TEST AUTOMATION ACCELERATORS

TEST AUTOMATION FRAMEWORK AS A SERVICE

TAFaaS

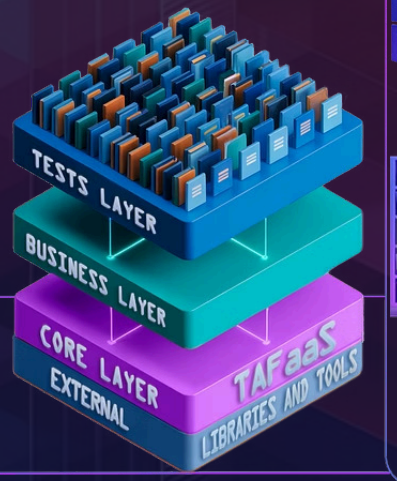
TEST AUTOMATION MANAGEMENT SYSTEM

TEST AUTOMATION FRAMEWORK LAYERS

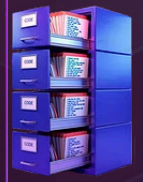
PROJECTS-SHARED
ARCHITECTURAL PATTERNS

PROJECT-SPECIFIC

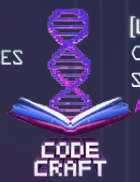
PROJECTS-REUSABLE



TEST AUTOMATION FRAMEWORK as a SERVICE

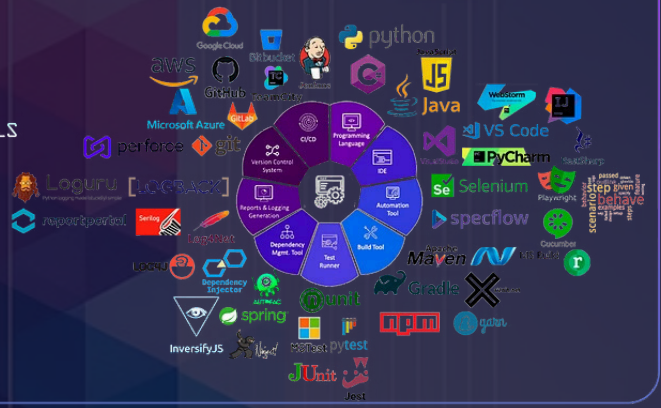


[LEVEL 1.1] ORGANIZATIONAL
STRUCTURED CATALOG OF REUSABLE USE CASES
Implementation Use Cases:
Task Description + Code + Documentation
Modular Package Distribution Architecture



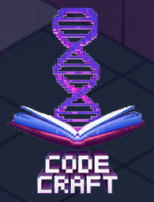
[LEVEL 1.2] ARCHITECTURAL
CODE OF ARCHITECTURAL RULES
SHARED PATTERNS/REUSABLE INTERFACES
Architectural Rules:
- Task Specification
- Standard Operating Procedures Manual
- Examples: Implementation Use Cases

[LEVEL 0] TECHNOLOGICAL
REUSABLE LIBRARIES AND TOOLS
- External (used AS IS)
- Enhanced External
- Advanced Company-Developed



[LEVEL 1.2] ARCHITECTURAL

CODE OF ARCHITECTURAL RULES
SHARED PATTERNS/REUSABLE INTERFACES



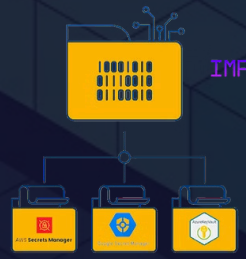
FUNDAMENTAL CRUD INTERFACES

ICreateSecret
IReadSecret
IUpdateSecret
IDeleteSecret

[LEVEL 1.1] ORGANIZATIONAL

STRUCTURED CATALOG OF REUSABLE USE CASES

Implementation Use Case:
Task Description + Code + Documentation
Modular Package Distribution Architecture



IMPLEMENTATION USE CASES CATEGORY: IMPLEMENT STORAGE OF SECRETS

IMPLEMENTATION USE CASE A) : IMPLEMENT STORAGE OF SECRETS IN AWS SECRETS MANAGER
IMPLEMENTATION USE CASE B) : IMPLEMENT STORAGE OF SECRETS IN GOOGLE SECRETS MANAGER
IMPLEMENTATION USE CASE C) : IMPLEMENT STORAGE OF SECRETS IN AZURE KEY VAULT

[LEVEL 0] TECHNOLOGICAL

REUSABLE LIBRARIES AND TOOLS

AWS Secrets Manager

Google Secret Manager

CURRENT APPROACH: UNMANAGEABLE

TAF as CLIENTS' PROJECTS INDIVIDUALLY DEVELOPED FEATURES

ALTERNATIVE APPROACH: MANAGEABLE

TEST AUTOMATION FRAMEWORK as a SERVICE (TAFaaS)

CURRENT APPROACH: UNMANAGEABLE

TAF as CLIENTS' PROJECTS

INDIVIDUALLY DEVELOPED FEATURES

ALTERNATIVE APPROACH: MANAGEABLE

TEST AUTOMATION FRAMEWORK as a SERVICE

TAFaaS

Decentralized Uncontrolled Development

by Single Person/Several People

Inidividual Uncontrollable Development

Usually Test Automation Framework is developed by single person (Lead/Architect), distinct and individual for different projects without any control or compliance oversight.

Each Lead/Architect is CEO of one's own

Leads/Architects operate with complete autonomy, leading to varied and often incompatible methodologies. So tens or even hundreds variations may exist.

"Free" Style Design

Each Lead or Architect, acting at their own discretion, designs the Test Automation Framework in one’s own preferred style, resulting in diverse and often incompatible approaches.

Centralized Supervised Development

by Test Automation Framework Engineers Team

Coordinated Controllable Development

Framework should be developed by a centralized team of Test Automation Framework Engineers, ensuring consistency and adherence to standards.

Collaborative Review Process

New implementations should undergo thorough review by a broader group of Test Automation Engineers to ensure usability and alignment with best practices.

Standardized Arhitecture Design Approach with Consistent Implementation Policies Across All Projects

Transition to a rigorously defined, "mathematically proved" architecture developed through a collaborative, community-driven process, ensuring consistency and robustness. All policies and updates are approved by a dedicated committee of Test Automation Framework Architects and the wider Test Automation Engineers Community, promoting a Standardized approach across all projects.

CURRENT APPROACH: UNMANAGEABLE

TAF as CLIENTS' PROJECTS

INDIVIDUALLY DEVELOPED FEATURES

ALTERNATIVE APPROACH: MANAGEABLE

TEST AUTOMATION FRAMEWORK as a SERVICE

TAFaaS

Individual Skillset Constrained

The design of the Test Automation Framework is often shaped and limited solely by the Lead or Architect's individual working experience and personal convictions, which may overlook broader industry best practices or more sophisticated solutions.

1 Lead/Architect

10 Years of Experience

5 Projects

Collective Knowledge & Experience Aggregated

Leveraging the combined knowledge of many Leads/Architects for Test Automation Framework Design and Development, fostering continuous improvement and innovation.

100 Leads/Architects

100 Leads/Architects × 10 Years of Experience each = 1000 Years of Combined Experience

100 Leads/Architects * 5 Projects of Experience each = 500 Total Projects Experience

CURRENT APPROACH: UNMANAGEABLE

TAF as CLIENTS' PROJECTS

INDIVIDUALLY DEVELOPED FEATURES

Mixed Practices Prevailed

Test Automation Frameworks often incorporate both effective and ineffective practices due to a lack of centralized oversight. The quality and adherence to best practices are limited by the varied skill sets of individual TAF features implementers and supporters.

ALTERNATIVE APPROACH: MANAGEABLE

TEST AUTOMATION FRAMEWORK as a SERVICE

TAFaaS

Best Practices /Fundamental Concepts Based

1

Propose & Evaluate Solutions, Collect User Experience

Multiple solutions and ideas are proposed, leveraging best practices from existing frameworks and deliberately excluding ineffective approaches. Eventually, as these solutions will be adopted by all users, collective engagement can be fostered in assessing their effectiveness, identifying shortcomings, and addressing any inconveniences encountered in Centralized Database of User Experience.

2

Open Contribution & Review

Any Test Automation Engineer can contribute, and all architectural solutions undergo detailed analysis for pros and cons, ensuring collaborative improvement.

3

"Mathematically Proved" Solutions

The optimal solution for each use case is chosen based on rigorous analysis and "mathematical proof", aiming for a single, standardized approach.

4

Continuous Regression & Refactoring

Regular regression analysis and re-evaluation ensure the TAFaaS adapts to technologies changes and new ideas, with refactoring implemented as needed to maintain high standards and address potential issues.

CURRENT APPROACH: UNMANAGEABLE

TAF as CLIENTS' PROJECTS

INDIVIDUALLY DEVELOPED FEATURES

ALTERNATIVE APPROACH: MANAGEABLE

TEST AUTOMATION FRAMEWORK as a SERVICE

TAFaaS

Project-Tailored

Single/Several Projects in Use

Usually each project has its own in some or other way distinct Test Automation Framework, though some same TAFs may spread across several projects in cases when Leads/Architects switch projects and use same or similar implementations.

Multiple Individual Frameworks per same Client

Even for a single client with multiple projects, each project often has its own individual Lead or Architect, resulting in distinct Test Automation Frameworks with little to no synchronization even within the same client, let alone across different clients.

Projects' Reusable

Uniformity and Reusability Accross All Possible Projects

The framework should be introduced to all possible projects. A centralized framework distribution ensures consistent quality and reusability across all projects.

CURRENT APPROACH: UNMANAGEABLE

TAF as CLIENTS' PROJECTS

INDIVIDUALLY DEVELOPED FEATURES

ALTERNATIVE APPROACH: MANAGEABLE

TEST AUTOMATION FRAMEWORK as a SERVICE

TAFaaS

Reinventing the Wheel

Same Wheel, New Blueprint

Lead/Architect often develops new Test Automation Framework or modify and reuse old one TAF from previous project(s) for each another one project in lead/next one project to lead.

Copy-Paste from Client to Client with Copyright Risks

Copy-pasting frameworks from Client to Client can lead to copyright violations.

Same Functionality - Duplicated Development

When a new feature like cloud provider integration is needed, every project has to implement it independently, wasting hundreds of hours company-wide from year to year.

Unified Invented

Company Implemented & Legally Protected

While a centralized team develops and supports the Test Automation Framework as a Service, it ensures legal compliance for the Company.

Request-Driven Functionality

New features are implemented once by the central team upon request, becoming available to all Projects, building a vast, reusable codebase, minimizing duplicated efforts and maximizing efficiency through a unified approach to test automation.

CURRENT APPROACH: UNMANAGEABLE

TAF as CLIENTS' PROJECTS

INDIVIDUALLY DEVELOPED FEATURES

ALTERNATIVE APPROACH: MANAGEABLE

TEST AUTOMATION FRAMEWORK as a SERVICE

TAFaaS

Isolated & Inefficient

Rarely Audited

Test Automation Frameworks live their own life leading to the implementation of numerous inefficient or even bad practices that go unchecked due to lack of time and/or other resources.

Single Programming Language Bound

Frameworks are typically tightly coupled to a single programming language, severely limiting their portability and reusability when projects transition to different tech stacks.

Poor AI Integration as consequence of Hundreds of Frameworks

At current moment, poor utilization of AI takes place. Each Project is forced to create own AI Wheel targeted at/customized for own separate Test Automation Framework , leading to redundant efforts and inconsistent results (e.g., 100 projects = 100 distinct AI modelling).

Integrated & Efficient

Constantly Audited

Implementation of manual peer reviews and automated auditing systems should ensure strict adherence to best practices, preventing the accumulation of technical debt and inefficiencies.

Cross Programming Languages Core

A framework built on language-agnostic concepts and design patterns allows for automatic translation into multiple programming languages, significantly enhancing portability and broader support.

Strong AI Integration having Single Framework

A centralized Test Automation Framework as a Service eliminates the need for redundant AI development. A robust core AI model can be trained on vast amounts of data sets and customized for overall architecture, with project-specific customization for non-core functionalities.

CURRENT APPROACH: UNMANAGEABLE

TAF as CLIENTS' PROJECTS

INDIVIDUALLY DEVELOPED FEATURES

ALTERNATIVE APPROACH: MANAGEABLE

TEST AUTOMATION FRAMEWORK as a SERVICE

TAFaaS

Poorly or Not Documented At All with Enduring Learning Curve

Ad-Hoc & Unstructured or Lack of Documentation

Documentation is often an afterthought and rarely well-written and poorly supported due to lack of time, relying on informal notes or individual memory, making it difficult for new team members to get up to speed or for existing members to troubleshoot.

Raised Onboarding Threshold

Every new project feels like a "new universe" due to a lack of standardized documentation, leading to additional ramp-up time for new Test Automation Engineers.

Knowledge Silos

Critical insights and solutions remain isolated within individual projects or even within specific team members, hindering knowledge sharing and collective growth.

Well-Documented with Progressive Learning Curve

Centralized Documentation and Learning Base

A single, continuously updated documentation and knowledge base serves as the definitive resource for all Test Automation Engineers.

Continuous Training Mechanisms

Regular training sessions, workshops, and video tutorials ensure that all Test Automation Engineers are proficient with the latest framework features and best practices.

Embedded Knowledge Checks

Integrated quizzes and practical exercises within documentation help reinforce learning and confirm comprehension, ensuring a consistent skill level.

Seamless Project Transitions

Test Automation Engineers can easily switch between projects, as the consistent and comprehensive documentation available for Test Automation Framework as a Service, so as previously continually gained usage exprerience minimize the learning curve and maximize productivity.

Writing new framework from scratch

Copy-paste from previous client

CURRENT APPROACH: UNMANAGEABLE

TAF as CLIENTS' PROJECTS

INDIVIDUALLY DEVELOPED FEATURES

BUSINESS FLOW A-current) : New Project "N" is launched (Internal/External with no previous development)

When a new Project "N" is launched and a team is gathered, a Lead/Architect is assigned, triggering one of two problematic scenarios for Test Automation Framework (TAF) setup. Both approaches lead to significant issues.

Use Case A) Write New Test Automation Framework

New Test Automation Framework is introduced by writing Core Functionality from scratch based on previous experience.

Use Case B) Copy-Paste Test Automation Framework from previous Client

New Test Automation Framework is introduced by copying from Old “Project O” where Lead/Architect worked before (sometimes as is, sometimes with namespaces changes, sometimes by additionally changing names of some classes).

These inefficient processes are the root cause of several critical problems:

1

Delayed Testing Start

New TAF development can take **1-4 sprints**, causing a significant time lag before the first test can be written.

2

Reinvention of the Wheel

Teams are forced to repeatedly build or adapt Core TAF functionality, both code and documentation, leading to duplicated efforts.

3

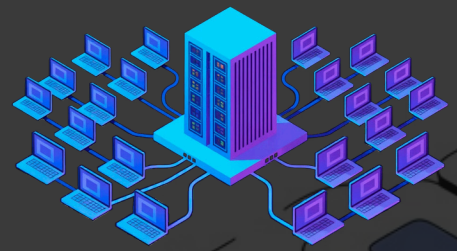
Copyright Violation Risk

Copying TAFs from Client to Client Projects carries the inherent risk of violating intellectual property rights.

4

Additional Learning Time

Junior, Middle, and Senior Test Automation Engineers must learn the unique principles of each Lead/Architect’s individually developed or copied Test Automation Frameworks.



ALTERNATIVE APPROACH: MANAGEABLE

TEST AUTOMATION FRAMEWORK as a SERVICE

TAFaaS

BUSINESS FLOW A-alternative) : New Project is launched (Internal/External with no previous development)

The core Test Automation Framework components are centrally developed and maintained in a Company repository, accessed as referenced packages. This fundamental shift unlocks several key benefits:

Instant Project Kickoff

Testing can begin almost immediately. The average goal for repository, CI/CD, and reporting configurations is 1-2 days, ideally within 4-8 hours after gaining all necessary client system accesses.

No Reinvention of the Wheel Necessary

There is no need to copy, rewrite, or create new code or documentation. Company-managed components and existing, comprehensive documentation are readily available for immediate use.

Company-Owned Assets

All codebase and documentation are centrally owned and managed by the company, ensuring consistency, quality, and legal compliance across all projects.

Reduced Onboarding Learning Curve

Test Automation Engineers may already be familiar with the company-managed Test Automation Framework from previous projects or internal training courses, accelerating onboarding.

CURRENT APPROACH: UNMANAGEABLE

CLIENTS' PROJECTS

INDIVIDUALLY DEVELOPED FEATURES

EVOLVE

ALTERNATIVE APPROACH: MANAGEABLE

TEST AUTOMATION FRAMEWORK as a SERVICE

TAFaaS

BUSINESS FLOW B): Existing Project in Development - Internal/External Company Managed from Start

When a Lead/Architect is changed in an existing project, the approach to the Test Automation Framework (TAF) may also become subject to change.

The new Lead/Architect may either:

1 Continue using Legacy Test Automation Framework of previous Lead/Architect

- **Lack of Documentation:** The legacy framework may have poor or outdated documentation, making it difficult for the new team to understand or extend it.
- **Technical Debt**: Older frameworks often accumulate hacks, workarounds, and deprecated practices that hinder scalability and maintainability.

2 Introduce a New Second Individually Developed Test Automation Framework due to perceived issues with the Legacy Test Automation Framework and Test Automation Engineers then begin writing tests in this new framework.

- **Reintroduces same issues of when New Project "N" is launched:** This scenario mirrors some of the problems faced as like when starting a new framework from scratch (reinvention of the wheel, copyright violation risk, additional learning time etc.).
- **Dual Framework Support:** The project must now support and maintain knowledge for **two** distinct TAFs, leading to increased complexity and overhead.

The new Lead/Architect continues to use the existing, Company-Managed Standardized Test Automation Framework as a Service, leveraging established best practices.

Higher Harmonization: Ensures consistency across projects, as new Leads/Architects are already familiar with the centralized standard.

Issue Avoidance: Many common problems associated with framework changes (e.g., knowledge transfer gaps, duplicate efforts) are avoided.

Continuous Improvement: Benefits from ongoing central development and support, rather than isolated, project-specific efforts.

CURRENT APPROACH: UNMANAGEABLE

CLIENTS' PROJECTS

INDIVIDUALLY DEVELOPED FEATURES

ALTERNATIVE APPROACH: MANAGEABLE

TEST AUTOMATION FRAMEWORK as a SERVICE

TAFaaS

BUSINESS FLOW C): Existing Project in Development - Client switched Vendors

When an existing project in development is transferred by the Client to the Company from other Vendor, Test Automation Framework may also become subject to change.

The Lead/Architect may either:

1 **Continue using Legacy Test Automation Framework** of previous Vendor

- **Lack of Documentation:** The legacy framework may have poor or outdated documentation, making it difficult for the new team to understand or extend it.
- **Technical Debt**: Older frameworks often accumulate hacks, workarounds, and deprecated practices that hinder scalability and maintainability.

2 **Introduce a New Second Individually Developed Test Automation Framework** due to perceived issues with the Legacy Test Automation Framework and Test Automation Engineers then begin writing tests in this new framework.

- **Reintroduces same issues of when New Project "N" is launched:** This scenario mirrors some of the problems faced as like when starting a new framework from scratch (reinvention of the wheel, copyright violation risk, additional learning time etc.).
- **Dual Framework Support:** The project must now support and maintain knowledge for **two** distinct TAFs, leading to increased complexity and overhead.

The Lead/Architect may either:

1 **Continue using Legacy Test Automation Framework** of previous Vendor

- **Lack of Documentation:** The legacy framework may have poor or outdated documentation, making it difficult for the new team to understand or extend it.
- **Technical Debt**: Older frameworks often accumulate hacks, workarounds, and deprecated practices that hinder scalability and maintainability.

2 **Introduce a New Second Company-Managed Standardized Test Automation Framework as a Service,** leveraging established best practices. Prior thorough assessment of the Legacy Test Automation Framework should be done, focusing on its criteria, functionality, and overall maintainability. If the framework has accumulated suboptimal practices over the years resulting in increased complexity, reduced efficiency, and difficulty in support, then evaluate the potential benefits of transitioning to Test Automation Framework as a Service.

1) Refactor Legacy and leave TAFaaS *as Only One* solution.

2) Add TAFaaS *as Another One* solution:

- **Dual Framework Support:** The project must now support and maintain knowledge for **two** distinct TAFs, leading to additional complexity and overhead. However, if correct evaluation was done, then time savings may pay off.

CURRENT APPROACH: UNMANAGEABLE

CLIENTS' PROJECTS INDIVIDUALLY DEVELOPED FEATURES

COSTS ALLOCATION ON THE CLIENT SIDE

As per the current approach, clients cover the expenses related to the initial development and ongoing support of the Core Test Automation Framework.

Initial Test Automation Framework Introduction

Clients pay for **1-4 sprints** of Lead/Architect work for the initial Core Test Automation Framework setup and introduction.

Additional Features/Add-ons Development

Any new features or add-ons required for the Core Test Automation Framework are paid for by the client as separate features/stories/tasks developed.

ALTERNATIVE APPROACH: MANAGEABLE

TEST AUTOMATION FRAMEWORK as a SERVICE

TAFaaS

COSTS ALLOCATION ON THE COMPANY SIDE

MONETARY/NON-MONETARY COMPENSATION CHANNELS

With a centralized framework, the company absorbs upfront costs, compensated through several strategic channels:

Internal Projects Costs Duplication Avoidance

Test Automation Framework as a Service may enable reusability and cost minimization across Internal Projects by eliminating the need to repeatedly invest in developing a core TAF parts foreach new individual project launched.

Fixed-Price Projects Costs Minimization

Test Automation Framework as a Service may provide advantage in fixed-price projects biddings by diminishing development time and costs for Test Automation, thus attracting additional Clients.

Presales Demo

Test Automation Framework as a Service may serve as a powerful Presales demonstration tool, allowing for quick demo deployments and first test runs to secure new deals and increase Client acquisition. The final goal is up to 4-8 hours for full Test Automation Framework as a Service deployment and first demo test results while negotiations take place.

Shared Reusable Libraries/Packages with Developers

Shared reusable libraries among Developers and Test Automation may help minimize costs by reducing duplicate coding efforts, so as accelerating project timelines.

1 Business Day Deployments

Test Automation Framework as a Service should enable rapid deployment capabilities, allowing for Core Test Automation Framework setup and initial testing boost within just one business day, providing competitive advantages and Client satisfaction for no need to wait and spend 1-4 sprints.

Crowd Testing Costs Optimization

Test Automation Framework as a Service may provide advantage in crowd testing projects by diminishing development time and costs for Test Automation, thus attracting additional Clients.

Enhanced Client Value

Clients receive more value for the same budget. Instead of rebuilding frameworks and reinventing the wheels, teams can focus on advanced features, like AI, or broader test coverage, improving quality and efficiency. The Company earns the similar, but delivers smarter work.

Distributed Cost Model

Development and support costs are spread across all projects, decreasing the per-project cost as adoption grows.

FINANCIAL INVESTMENTS

STRATEGIC ADVANTAGES

PROJECT MANAGEMENT: CLIENT EXPECTATIONS

GENERAL	$\frac{\text{WORK}}{\text{TIME} * \text{COSTS}} * \text{QUALITY} \rightarrow ?$
TAFaaS$_{\text{SHOULD PROVIDE}}$	$\frac{\text{WORK}_{\text{MAXIMIZED}}}{\text{TIME}_{\text{MINIMIZED}} * \text{COSTS}_{\text{MINIMIZED}}} * \text{QUALITY}_{\text{MAXIMIZED}} \rightarrow$ PROJECT MANAGEMENT EFFECTIVENESS $+\infty_{\text{MAXIMIZED}}$
OTHERS	$\frac{\text{WORK}_{\text{MINIMUM}}}{\text{TIME}_{\text{MAXIMUM}} * \text{COSTS}_{\text{MAXIMUM}}} * \text{QUALITY}_{\text{MINIMUM}} \rightarrow$ PROJECT MANAGEMENT EFFECTIVENESS $-\infty_{\text{MINIMIZED}}$

TAF as CLIENTS' PROJECTS INDIVIDUALLY DEVELOPED FEATURES

WORK EFFICIENCY MINIMIZED

ISOLATED TEST AUTOMATION FRAMEWORKS CONTAIN ONLY
THE USE CASES OF CURRENT PROJECT,
BASED ON ITS SOLE PROJECT MANAGEMENT HISTORY,
COVER ONLY PRESENT NEEDS,
AND REQUIRE ADDITIONAL DEVELOPMENT OF CORE
FUNCTIONS IF SUCH NEEDS ARISE IN THE FUTURE.

TIME ALLOCATION MAXIMIZED

ISOLATED TEST AUTOMATION FRAMEWORKS MAY TAKE
SPRINTS/WEEKS FOR INITIAL DEVELOPMENT
AND PERIODICAL ITERATIONS
LATER FOR PROVIDING SUPPORT
OF ADDITIONAL CORE FUNCTIONALITY
THE NEED OF WHICH MAY ARISE OVER TIME.

TEST AUTOMATION FRAMEWORK AS A SERVICE

WORK EFFICIENCY MAXIMIZED

TAFaaS AGGREGATES MANY YEARS OF EXPERIENCE OF WORK
AND CONTAINS AT THE VERY LEAST
ALL MOST WIDELY USED USE CASES
ACROSS ALL PROJECTS
THROUGHOUT ALL PROJECT MANAGEMENT HISTORY
AVAILABLE FOR BOTH PRESENT AND FUTURE PROJECTS NEEDS.

TIME ALLOCATION MINIMIZED

TAFaaS PROVIDES OUT OF THE BOX SOLUTION FOR CORE CODE BASE.
NO NEED TO SPEND SPRINTS/WEEKS FOR INITIAL DEVELOPMENT
AND LATER FOR ADDING SUPPORT OF ADDITIONAL FUNCTIONALITY,
CAUSE IT MAY BE ALREADY SUPPORTED BY TAFaaS.
REFERENCE CODE LIBRARY AND USE IT IN A MINUTE.
THE TIME WILL BE REQUIRED ONLY
FOR PROJECT-SPECIFIC IMPLEMENTATIONS.

TAF as CLIENTS' PROJECTS INDIVIDUALLY DEVELOPED FEATURES

COSTS ASSIGNMENT MAXIMIZED

ISOLATED TEST AUTOMATION FRAMEWORKS TAKE TIME
AND AS A RESULT BEAR COSTS FOR EACH CLIENT
FOR INITIAL DEVELOPMENT AND PERIODICAL ITERATIONS LATER
FOR PROVIDING SUPPORT OF ADDITIONAL CORE FUNCTIONALITY
LEARNING CURVE COSTS, COSTS ON TAILOR-MADE AI INTEGRATIONS.

QUALITY ASSURANCE MINIMIZED

ISOLATED TEST AUTOMATION FRAMEWORKS OFTEN REFLECT
THE INDIVIDUAL PRACTICES
OF SEPARATE ARCHITECTS/TEST AUTOMATION ENGINEERS,
WHICH ARE NOT ALWAYS ALIGNED WITH BEST STANDARDS,
BUT RATHER EVOLVE OVER TIME FROM PERSONAL EXPERIENCE
GAINED ON SPECIFIC PROJECTS THROUGHOUT THEIR CAREERS

TEST AUTOMATION FRAMEWORK AS A SERVICE

COSTS ASSIGNMENT MINIMIZED

TAFaaS WHILE PROVIDING OUT OF THE BOX CORE CODE BASE
MINIMIZE COSTS OF CLIENTS
FOR INITIAL DEVELOPMENT AND FURTHER PERIODICAL ITERATIONS
FOR PROVIDING SUPPORT OF ADDITIONAL CORE FUNCTIONALITY,
LESS LEARNING CURVE COSTS, LESS COSTS ON AI INTEGRATIONS
OF STRANDARDIZED SOLUTIONS.
COSTS ARE DISTRIBUTED ACROSS MANY PROJECTS:
THE MORE PROJECTS INVOLVED THE LESS COSTS PER PROJECT.

QUALITY ASSURANCE MAXIMIZED

TAFaaS IS BASED AND EVOLVES ON THE APPROVED
BEST OF THE BEST PRACTICES
GATHERED WITHIN INTEGRATED COLLABORATIVE SYSTEM
FROM ALL PROJECTS FOR ALL THE TIME,
PROVEN BY THE TIME OF ALL PROJECT MANAGEMENT HISTORY
AND EXPERIENCE OF NUMEROUS TEST AUTOMATION ENGINEERS

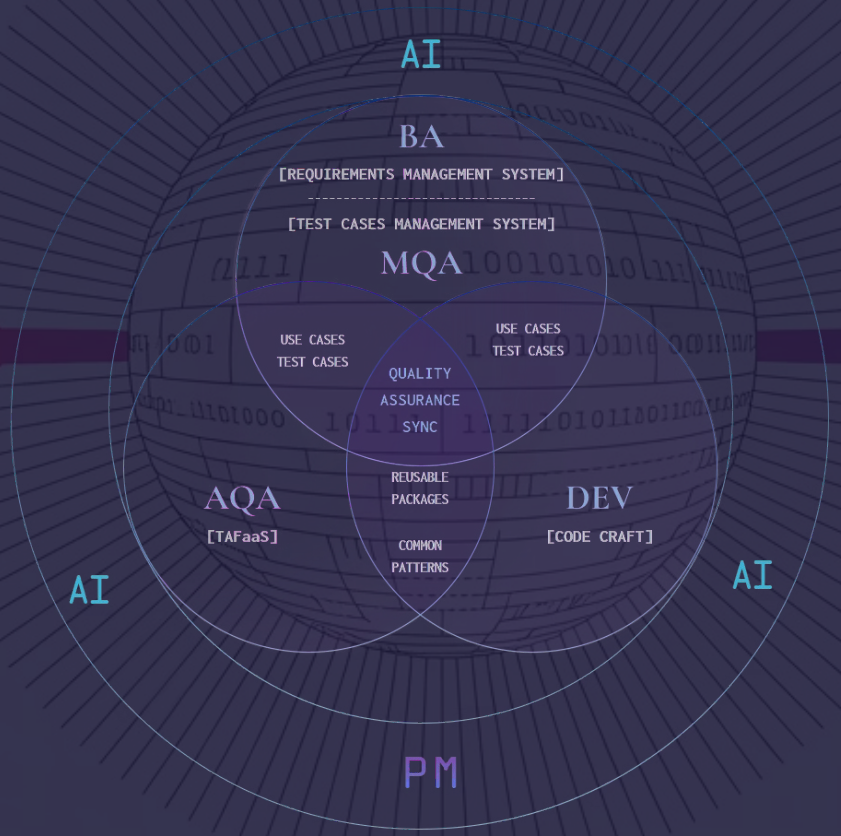
TEST AUTOMATION FRAMEWORK as a SERVICE (TAFaaS)

AS INTEGRAL PART OF

PROJECT MANAGEMENT SOFTWARE DEVELOPMENT LIFECYCLE (SDLC)

THAT PROVIDES

WORK$_{MAXIMIZED}$ TIME$_{MINIMIZED}$ COSTS$_{MINIMIZED}$ QUALITY$_{MAXIMIZED}$



QUALITY ASSURANCE MANAGEMENT ECOSYSTEM

TEST AUTOMATION FRAMEWORK AS A SERVICE
MANAGEMENT AND BUDGETING

THE BOARD OF ARCHITECTS
CODE NAME: THE SUPREME COURT






ARCHITECTS
Responsible for ARCHITECTURAL JUDGEMENTS. At least 1 EXPERT PER PROGRAMMING LANGUAGE.

CODE NAME: JUDGES
Preside over court proceedings, ensure fairness, interpret and apply the law, and issue rulings or sentences.

COSTS COVERAGE: Architects MUST work on PRODUCTION Projects. Workload ratio may vary.
For example: 75% Production Activities / 25% of TAFaaS Activities.

ADDITIONAL MUST APPLIED RULES:
Every 1-2 Years there should be rotation of TAFaaS Architects between PRODUCTION Projects, which should help to fulfill at least the following functional requirements:

1) Audit and Test Functions: Architects will be able to Imply and Test the correctness of usage of Core TAFaaS Code Base within PRODUCTION ENVIRONMENTS.
2) Framework Improvements Function: by gaining Cross-Project/Cross-Domain Experience for different types of real life implications/use cases, generalization of knowledge by Architects can be done and better understanding of vectors of Test Automation Framework as a Service improvements.

REVIEWERS
Besides THE JUDGES are responsible for approving VERDICTS on CHANGES, PULL REQUESTS.

CODE NAME: TRIAL JURY OF PEERS
Responsible for listening to evidence, evaluating facts, applying the judge's instructions on the law, deliberating together, and ultimately reaching and returning a verdict (liable/not liable).

COSTS COVERAGE: Jurors may be compensated modestly, but Jury Duty is considered a civic obligation, not a paid job.

BENCH POOL
Responsible for providing TEST AUTOMATION FRAMEWORK AS A SERVICE SUPPORT: NEW IMPLEMENTATIONS, REFACTORINGS.

CODE NAME: BAILIFFS
A uniformed Court Officers responsible for maintaining order, providing security, and assisting the Judges and Jury during court proceedings.

COSTS COVERAGE: THE BENCH is covered at the costs of Central Government Expenses, which should not add any additional costs to the budget of Test Automation Framework as a Service itself.

USERS
Test Automation Engineers on Production Projects to use Test Automation Framework as a Service.

CODE NAME: WITNESSES
Provide testimony relevant to the case; can be called by either side.

COSTS COVERAGE: 100% by Production

SUCH STRUCTURE SHOULD MINIMIZE THE BUDGETING OF TEST AUTOMATION FRAMEWORK AS A SERVICE

Implementation Map

Stage 1: Analytics

Analyze available Internal Projects code base on Core Test Automation Framework implementation variations for single/several Programming Language(s)

Stage 2: TAFaaS approach Alpha Testing

Core Test Automation as a Service approach base implementation (POC) testing on one/several Lab/Internal Projects for single/several Programming Language(s).

Stage 3: TAFaaS approach Beta Testing

Core Test Automation as a Service approach base implementation (POC) testing on one/several Clients' Projects for single/several Programming Language(s).

Stage 4: Additional Development, wider Adoption and Support of TAFaaS approach

- **Gathering additional analytics**
- **Developing additional Core Functionality**
- Support of several Programming Languages
- **Creating and refining documentation**
- **Developing Learn Portal courses**
- **Introducing Presales Demo procedures**
- **Adoption across Projects**